fapolicyd for Power Users

Leave able to configure and debug it

SELinux Development Team Workshop

October 20, 2026

Goal of this talk

After these slides (+ live demo) you can:

- Install and verify fapolicyd
- Trust unpackaged software
- Add a `rules.d` drop-in safely
- Debug a denial and triage AVC vs FANOTIFY

How we get there

- **Why** — when to use it (vs SELinux)
- **What** — trust DB, rules, integrity
- **Configure** — exact workflows
- **Live** — one end-to-end example

At home = practice, not first contact

- Config knowledge is *in this deck*
- `EXERCISES.md` drills muscle memory
- Showcases = present later, not learn first

Agenda

Why

What you configure

Configure

Live

You leave able to

SELinux asks

Given labels, is this access allowed?

Behavior · domain / type / MLS

fapolicyd asks

Is this file a known artifact?

Trust · RPM / manual provenance

Use both

- SELinux: how it behaves
- fapolicyd: whether it is known
- Same action can hit both denials

What you gain

- Block home / tmp unpackaged code
- Stop ld.so / interpreter smuggling
- Whitelist by trust, not by labels alone

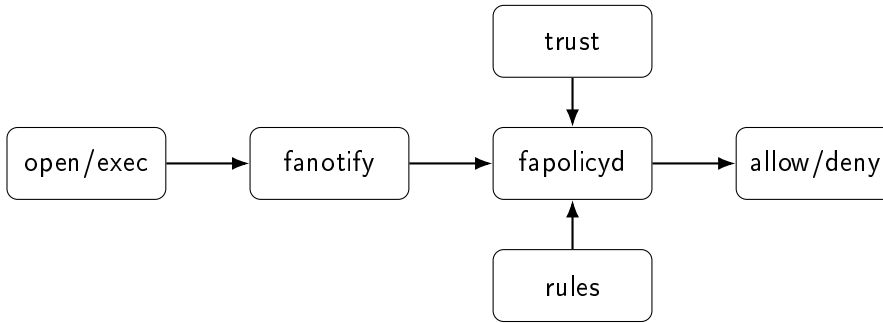
Honest limits

- Root can disable it
- Same class as SELinux permissive
- Not a substitute for MAC

Three places that matter

1. Trust DB — known files
2. `rules.d/` — policy
3. `fapolicyd.conf` — mode / integrity / tuning

Architecture (for intuition)



Hard rule

Never strace / gdb the daemon

SIGSTOP $\Rightarrow$ system-wide deadlock

Trust — two backends

- `rpmdb` — from RPM (auto via plugin)
- `filedb` — you add in `trust.d/`

Trust — entry

```
filedb /opt/app/bin/x  SIZE  SHA256
```

Rules see `trust=1` or `trust=0`

Rules — recipe

```
decision permission subject :  object
```

First match wins

Rules — where they live

- Edit `/etc/fapolicyd/rules.d/NN-name.rules`
- `fagenrules` builds `compiled.rules`
- Never hand-edit compiled output

Default policy

- Shipped set: `known-libs`
- Trusted stuff mostly works
- Unpackaged execute/open of code → deny

Integrity in conf

- `none` — default
- `size` — good first step
- `sha256` — costly; plan capacity

Workflow A — install

1. `dnf install fapolicyd fapolicyd-selinux rpm-plugin-fapolicyd`
2. Ensure ≥ 1 audit rule exists
3. `systemctl enable -now fapolicyd`

Workflow A — verify

- `fapolicyd-cli -check-status`
- `fapolicyd-cli -list | head`
- Expect: Permissive false, trust entries > 0

Audit rule (why)

- Without any audit rule, `deny_audit` is quiet
- Dummy: `auditctl -w /etc/shadow -p w`
- Watch $\neq$ chmod — permissions unchanged

Workflow B — trust an /opt app

This is the default remediation.

Trust /opt — commands

```
fapolicyd-cli -file add /opt/myapp \
  -trust-file myapp
fapolicyd-cli -update
```

Trust /opt — remember

- `-file add` writes `trust.d/`
- `-update` reloads the daemon DB
- Prefer this over allow rules

Inspect trust

```
fapolicyd-cli -dump-db | grep myapp
ls /etc/fapolicyd/trust.d/
```

Workflow C — custom rule

Use only when you need *who* / *how*,
not just “is it known?”

Custom rule — drop-in

```
# /etc/fapolicyd/rules.d/55-ping.rules
allow perm=any gid=wheel :
  trust=1 path=/usr/bin/ping
deny_audit perm=execute all :
  trust=1 path=/usr/bin/ping
```

Custom rule — apply

```
fagenrules
fapolicyd-cli -check-rules -lint
systemctl reload fapolicyd
fapolicyd-cli -list | grep ping
```

Custom rule — safety

- Number file so it sits *above* deny-all
- Bad reload keeps previous ruleset
- Keep drop-ins sparse

Workflow D — conf knobs

File: `/etc/fapolicyd/fapolicyd.conf`

Conf — soak then enforce

- `permissive = 1` while learning / soaking
- Watch denials, fix trust/rules
- Then `permissive = 0` + restart

Conf — integrity

- Start with `integrity = size` if you want more
- Avoid jumping straight to sha256

Decision tree — denied?

1. `ausearch -m avc` → SELinux
2. `ausearch -m fanotify` → fapolicyd
3. Neither / unclear → debug-deny

If FANOTIFY deny

- Should be known software? → trust + `-update`
- Who/how exception? → `rules.d` drop-in

Debug loop (1/2)

1. `systemctl stop fapolicyd`
2. `fapolicyd -permissive -debug-deny`
3. Reproduce → `rule:N`

Debug loop (2/2)

4. `fapolicyd-cli -list` → map N
5. Fix trust or move allow above deny
6. Restart service when done

CLI cheat — inspect

- `-check-status`
- `-list`
- `-dump-db`

CLI cheat — change

- `-file add/delete/update`
- `-update`
- `-check-rules -lint`

Anti-patterns

- Allow-list sprawl instead of trust
- Edit `compiled.rules`
- Assume SELinux allow $\Rightarrow$ fapolicyd allow

We configure together

Install → deny clone of `ls` → trust it → allow

Setup

- `sudo demos/00-lab-prep.sh`
- Or install + enable by hand

Deny

```
cp /usr/bin/ls /opt/fapo-workshop/my-ls
chmod +x /opt/fapo-workshop/my-ls
/opt/fapo-workshop/my-ls          # DENY
```

Configure fix

```
fapolicyd-cli -file add \
  /opt/fapo-workshop/my-ls -trust-file workshop
fapolicyd-cli -update
/opt/fapo-workshop/my-ls          # ALLOW
```

You just configured trust

- Diagnosed untrusted execute
- Applied the standard fix
- Verified allow

Checklist

- ☐ Install + verify status
- ☐ Trust an `/opt` tree
- ☐ Add / reload a rules drop-in
- ☐ Debug with `rule:N`
- ☐ Triage AVC vs FANOTIFY

Practice next

- `EXERCISES.md` — predict / do / answer
- `TAKE-HOME.md` — cookbook reference
- `demos/showcases/` — demo to others later

Discussion

- Where on staff machines?
- How do we teach AVC vs FANOTIFY?

References

- linux-application-whitelisting/fapolicyd
- RHEL Security hardening — fapolicyd
- man: fapolicyd(8), rules(5), cli(8)

Questions

You should be able to configure it now.

Drill with `EXERCISES.md`